

DevRecall

1. Идея проекта

Нужно разработать персональное приложение для подготовки разработчика к техническим собеседованиям.

Основной формат — карточки вопросов и ответов с системой повторений.

Приложение должно ощущаться как **нормальная небольшая мобильная программа**, а не как страница сайта с вопросом и четырьмя кнопками.

Основной способ использования:

- iPhone → Telegram → DevRecall;
- Windows → Telegram Desktop → DevRecall;
- при разработке приложение должно также открываться обычной ссылкой в браузере.

Главная задача приложения:

помочь системно изучать большой объём теории, запоминать её, возвращать забытые вопросы на повторение и видеть собственный прогресс подготовки к собеседованиям.

Это личный инструмент подготовки, а не коммерческий SaaS.

Поэтому архитектура должна быть аккуратной, но без enterprise-overengineering.

2. Основной сценарий пользователя

Пользователь открывает Telegram.

Заходит в бота DevRecall.

Нажимает:

«Начать обучение»

Открывается Telegram Mini App.

На главной странице пользователь видит:

- сколько карточек нужно повторить сегодня;
- сколько новых карточек доступно;
- серию дней обучения;
- общий прогресс;

- темы подготовки;
- кнопку «Продолжить обучение».

Дальше:

Главная → Обучение → Вопрос → Попытка вспомнить → Показать ответ → Оценить свой ответ → Следующая карточка.

Пример:

JavaScript · Event Loop

Что такое microtask queue и когда она выполняется?

Кнопка:

Показать ответ

После этого отображается нормальный подробный ответ, включая текст и при необходимости код.

Внизу:

- Не знаю
- Сложно
- Нормально
- Легко

Выбор влияет на дату следующего повторения.

3. Требования к интерфейсу

Интерфейс должен быть mobile-first.

Нельзя делать ощущение дешёвого HTML-прототипа.

Нужен визуальный стиль небольшой современной мобильной программы:

- аккуратная типографика;
- нормальные отступы;
- скруглённые карточки;
- понятная визуальная иерархия;
- крупные touch-target кнопки;
- фиксированная нижняя навигация;
- поддержка safe-area iPhone;
- плавные, но минимальные анимации;
- тёмная и светлая тема;
- адаптация под цвета Telegram;

- хорошая работа на desktop;
- максимальная ширина центрального контента на desktop примерно 700–900px.

Не использовать тяжёлый UI framework только ради готовых компонентов.

Разрешена небольшая библиотека иконок.

Предпочтительно:

lucide-vue-next

Основной интерфейс написать самостоятельно.

4. Навигация

Основная нижняя навигация:

Главная

Home

Темы

Topics

Избранное

Favorites

Статистика

Statistics

Отдельно:

Настройки

Settings

Экран обучения должен быть отдельным полноэкранным режимом.

Во время обучения нижнюю навигацию можно скрывать, чтобы пользователь не отвлекался.

5. Главная страница

Главная должна сразу отвечать на вопрос:

Что мне сегодня учить?

Пример структуры:

Верхняя часть

Приветствие.

Например:

Добрый вечер 🖐️

Ниже:

Сегодня к повторению: 18 карточек

Главная СТА-кнопка:

Начать повторение

Если пользователь уже начал сессию:

Продолжить обучение

Краткая статистика

Несколько небольших карточек:

🔥 6 дней
Серия

32
Пройдено сегодня

184
Изучено

Общий прогресс

Например:

Подготовка к собеседованию

38%

Progress bar.

Темы

Небольшой список основных тем:

JavaScript — 43%

Vue — 37%

PHP / Laravel — 26%

SQL — 51%

RAG / LLM — 18%

и т.д.

Есть кнопка:

Все темы

6. Темы обучения

На первоначальном этапе создать следующие направления.

Frontend

- JavaScript
- Vue 2
- Vue 3
- Browser / Web
- HTML / CSS

Backend

- PHP
- Laravel
- REST API
- архитектура backend

Database

- SQL
- PostgreSQL
- MS SQL
- индексы
- транзакции
- оптимизация запросов

Infrastructure

- Git
- Docker
- CI/CD
- Kubernetes
- RabbitMQ
- MinIO / S3

AI

- RAG
- LLM
- embeddings
- pgvector
- vector search
- full-text search
- RRF
- reranking
- chunking
- Ollama

Security

- XSS
- stored XSS
- CSRF
- SQL injection
- JWT / tokens
- RBAC
- fail-closed
- безопасность API
- авторизация / аутентификация

Testing

- unit testing
- Vitest
- frontend testing

Моё резюме

Особый раздел.

Здесь должны находиться вопросы непосредственно по заявленному опыту.

Например:

- Контент-1;
- Комдив 2.0;
- RAG pipeline;
- OCR;

- CI/CD;
- Laravel 8 → 9;
- N+1;
- RBAC;
- visual BP constructor;
- RabbitMQ;
- WORM;
- MinIO;
- PostgreSQL RLS;
- безопасность загрузки файлов;
- архитектурные решения.

Этот раздел нужен для тренировки вопросов вида:

«У вас в резюме написано X. Расскажите подробнее».

7. Структура тем

У темы могут быть разделы.

Например:

JavaScript

- Fundamentals
- Scope
- Closure
- Event Loop
- Promise
- async/await
- Prototype
- this
- Arrays
- Objects
- Modules
- Memory
- Error handling

Vue

- Reactivity
- Lifecycle
- computed/watch
- components
- props/events
- slots
- Vue Router
- Vuex
- Pinia

- Composition API
- Vue 2 vs Vue 3
- performance

На странице темы показать:

- общий процент;
- количество карточек;
- количество новых;
- количество изученных;
- количество сложных;
- количество карточек на повторение;
- разделы темы.

Кнопка:

Учить тему

8. Структура карточки

Каждая карточка должна иметь стабильный ID.

Пример модели:

```
interface Card {  
  id: string;  
  
  topicId: string;  
  sectionId?: string;  
  
  question: string;  
  answer: string;  
  
  level?: "basic" | "middle" | "advanced";  
  
  tags?: string[];  
  
  order?: number;  
  
  enabled: boolean;  
}
```

Пример:

```
{  
  "id": "js-event-loop-001",  
  "topicId": "javascript",  
}
```



```
"sectionId": "event-loop",
"question": "Что такое microtask queue?",
"answer": "...",
"level": "basic",
"tags": ["event-loop", "promise"],
"enabled": true
}
```

ID карточки после создания нельзя менять без необходимости.

Это важно, потому что весь пользовательский прогресс будет привязан именно к `cardId`.

9. Формат ответов

Ответы должны поддерживать Markdown.

Нужно нормально отображать:

- обычный текст;
- списки;
- bold;
- inline code;
- code blocks;
- заголовки;
- небольшие таблицы при необходимости.

Код должен нормально читаться на мобильном экране.

Длинные строки кода не должны ломать интерфейс.

Контент необходимо безопасно рендерить.

10. Экран обучения

Основной экран приложения.

Наверху:

JavaScript · Event Loop

Дополнительно можно показывать:

12 / 20

или progress bar текущей сессии.

Карточка

Показывается вопрос.

Ответ изначально скрыт.

Также доступны:

- добавить/убрать из избранного;
- индикатор уровня вопроса;
- тема;
- раздел.

Основная кнопка:

Показать ответ

11. Оценка ответа

После показа ответа пользователь самостоятельно оценивает себя.

Четыре варианта:

Не знаю

Пользователь не смог нормально ответить.

Сложно

Что-то вспомнил, но ответ неполный или дался тяжело.

Нормально

Ответил в целом правильно.

Легко

Знает вопрос уверенно.

Внутренние значения:

```
type ReviewRating =  
  | "again"  
  | "hard"
```

```
| "good"  
| "easy";
```

12. Система повторения

Алгоритм обязательно вынести из UI в отдельный сервис.

Например:

```
calculateNextReview(progress, rating)
```

Первая версия алгоритма должна быть простой и понятной.

Не нужно сразу реализовывать полноценный Anki/FSRS.

Не знаю

Карточка должна появиться повторно ещё в текущей или ближайшей сессии.

Следующее основное повторение — примерно через 1 день.

Увеличивается `wrongCount`.

Сложно

Следующий интервал небольшой.

Пример:

```
предыдущий интервал × 1.5
```

Минимум 1 день.

Нормально

Интервал существенно увеличивается.

Например:

предыдущий интервал × 2.5

Минимум 3 дня.

Легко

Интервал увеличивается сильнее.

Например:

предыдущий интервал × 4

Минимум 7 дней.

В дальнейшем должна существовать возможность заменить этот алгоритм на FSRS без переписывания UI и базы.

13. Прогресс карточки

Пример модели:

```
interface CardProgress {
    cardId: string;

    status:
        | "new"
        | "learning"
        | "review"
        | "mastered";

    repetitions: number;

    correctCount: number;
    wrongCount: number;

    currentIntervalDays: number;

    lastRating?: ReviewRating;

    lastReviewedAt?: string;
    nextReviewAt?: string;
```

```
favorite: boolean;  
}
```

14. Определение сложных карточек

Приложение должно автоматически понимать, какие вопросы вызывают проблемы.

Например, карточка считается сложной, если:

- часто нажимается «Не знаю»;
- часто нажимается «Сложно»;
- высокий `wrongCount`;
- несколько раз подряд пользователь не может её закрепить.

На странице темы можно показывать:

Сложные вопросы — 12

В будущем можно добавить отдельный режим:

Повторить сложные

15. Избранное

На каждой карточке должна быть кнопка:

☆ / ★

Пользователь может пометить вопрос как избранный.

Раздел «Избранное» показывает все такие карточки.

Необходимо:

- фильтрация по теме;
- возможность открыть карточку;
- возможность удалить из избранного;
- возможность запустить:

Учить избранное

16. Режимы обучения

Не обязательно реализовывать всё одновременно, но архитектура должна их позволять.

Сегодня

Главный режим.

Сначала карточки, которые пора повторить.

Затем новые.

Учить тему

Пользователь выбирает конкретную тему.

Например:

SQL

Получает карточки только этой темы.

Учить раздел

Например:

JavaScript → Event Loop

Избранное

Только избранные карточки.

Сложные

Только проблемные вопросы.

Новые

Только ещё не изучавшиеся вопросы.

17. Логика формирования ежедневной сессии

При запуске «Начать обучение»:

1. собрать карточки с `nextReviewAt <= now`;
2. поставить их в начало;
3. после них добавить новые карточки;
4. учитывать установленный дневной лимит;
5. карточки с `again` могут повторно попасть в текущую сессию.

Необходимо избегать ситуации:

пользователь ошибся → карточка тут же показывается снова следующей.

Лучше вернуть её через несколько других вопросов.

Например через 5–10 карточек.

18. Завершение сессии

После окончания обучения показать экран результата.

Например:

Отлично

Сегодня пройдено: **34**

Не знаю: **5**

Сложно: **8**

Нормально: **17**

Легко: **4**

Новых карточек: **12**

Повторено: **22**

Время обучения: **18 минут**

Кнопки:

[На главную](#)

[Продолжить обучение](#)

19. Статистика

Нужен отдельный нормальный экран статистики.

Не надо превращать его в VI-систему.

Достаточно полезной статистики.

Сегодня

- сколько карточек пройдено;
 - сколько новых;
 - сколько повторено;
 - распределение Again / Hard / Good / Easy;
 - сколько минут занимался.
-

Неделя

Небольшой график по последним 7 дням.

Например:

ПН	32
ВТ	19
СР	41
...	

Можно реализовать простым CSS.

Не подключать большую chart-библиотеку только ради одного графика.

Серия

Например:

🔥 7 дней подряд

Серия увеличивается, если пользователь хотя бы немного занимался в конкретный день.

Общий прогресс

Показывать:

- всего карточек;
- новых;
- изучаемых;
- на повторении;
- хорошо закреплённых.

Прогресс по темам

Например:

JavaScript — 63%

Vue — 51%

SQL — 44%

Laravel — 37%

RAG — 21%

Сложные темы

В дальнейшем приложение может показать:

Больше всего ошибок:

1. SQL Transactions
2. JavaScript Event Loop
3. Laravel Container

20. Что считать изученной карточкой

Нужно различать:

видел карточку

и

действительно знаешь карточку.

Поэтому общий progress нельзя считать просто:

открыл / всего

Карточка считается начатой после первого review.

Карточка считается хорошо изученной, когда пользователь несколько раз успешно ответил и интервал повторения стал достаточно большим.

Точные параметры вынести в study service, чтобы их можно было менять.

21. История повторений

Для статистики желательно хранить каждую попытку пользователя.

Модель:

```
interface ReviewEvent {
  id: string;

  cardId: string;

  rating: ReviewRating;

  reviewedAt: string;

  previousIntervalDays: number;
  nextIntervalDays: number;
}
```

Это позволит позже строить нормальную статистику и улучшать алгоритм.

22. Добавление большого количества карточек

Это очень важное требование.

Не должно быть необходимости вручную создавать карточки через интерфейс приложения.

На первом этапе контент хранится отдельно от Vue-кода.

Структура примерно такая:

```
src/  
  content/  
  
    topics.json  
  
  cards/  
  
    javascript/  
      fundamentals.json  
      event-loop.json  
      promise.json  
      closure.json  
  
    vue/  
      reactivity.json  
      lifecycle.json  
      vuex.json  
      pinia.json  
  
    sql/  
      joins.json  
      indexes.json  
      transactions.json  
  
    laravel/  
      eloquent.json  
      middleware.json  
      queues.json  
  
    rag/  
      embeddings.json  
      pgvector.json  
      hybrid-search.json  
  
    resume/  
      content1.json  
      komdiv.json  
      architecture.json
```

Таким образом пользователь сможет сказать AI-агенту:

Добавь 50 карточек по SQL indexes.

Агент создаёт/обновляет соответствующий JSON.

Приложение автоматически видит новые карточки после следующего deploy.

23. Валидация карточек

Обязательно сделать скрипт проверки контента.

Например:

```
npm run validate:cards
```

Он должен проверять:

- уникальность `card.id`;
- наличие `topic`;
- существование `topicId`;
- существование `sectionId`;
- наличие `question`;
- наличие `answer`;
- корректный `level`;
- отсутствие пустых карточек;
- отсутствие очевидных дубликатов ID.

Если контент сломан, build должен сообщать понятную ошибку.

24. Массовое добавление карточек

Архитектура должна позволять агенту генерировать сотни карточек.

Например пользователь говорит:

Создай 100 вопросов по JavaScript уровня middle.

Агент:

1. смотрит существующие карточки;
 2. не создаёт дубликаты;
 3. распределяет вопросы по разделам;
 4. создаёт стабильные ID;
 5. добавляет вопросы;
 6. запускает validation;
 7. сообщает количество добавленных карточек.
-

25. Почему карточки сначала хранятся JSON

На первоначальном этапе база карточек не требует полноценной БД.

Даже несколько тысяч текстовых карточек спокойно могут быть частью приложения.

База понадобится не просто потому, что «закончилась память», а когда появится необходимость:

- редактировать карточки без нового deploy;
- добавлять карточки через интерфейс;
- генерировать их AI автоматически;
- делать административную панель;
- использовать приложение несколькими людьми;
- делать серверный поиск;
- версионировать контент;
- управлять десятками тысяч карточек.

До этого JSON значительно проще.

26. Очень важное архитектурное требование

Vue-компоненты не должны знать, откуда пришли карточки.

Создать интерфейс:

```
interface CardRepository {
  getTopics(): Promise<Topic[]>;

  getCards(): Promise<Card[]>;

  getCardsByTopic(
    topicId: string
  ): Promise<Card[]>;

  getCardById(
    cardId: string
  ): Promise<Card | null>;
}
```

На первом этапе:

```
StaticCardRepository
```

читает JSON.

Позже:

```
SupabaseCardRepository
```

читает PostgreSQL.

UI при этом менять не нужно.

27. Хранение пользовательского прогресса

В отличие от самих карточек, прогресс должен в итоге синхронизироваться между устройствами.

То есть:

iPhone

и

Telegram Desktop

должны показывать одинаковые:

- изученные карточки;
 - избранное;
 - статистику;
 - серию;
 - даты повторения.
-

28. Локальный режим разработки

Пока приложение запускается обычным браузером:

```
http://localhost:5173
```

можно использовать:

```
localStorage
```

или небольшой local repository.

Это позволит разрабатывать приложение без Telegram.

Создать:

```
ProgressRepository
```

и реализации:

```
LocalProgressRepository  
ApiProgressRepository
```

29. База данных

Для серверного хранения использовать:

Supabase PostgreSQL

Не использовать Firebase или собственный постоянно работающий backend на первом этапе.

30. Предварительная схема БД

users

```
id  
telegram_id  
username  
first_name  
created_at  
last_seen_at
```

card_progress

```
user_id  
card_id  
  
status  
  
repetitions  
  
correct_count  
wrong_count  
  
current_interval_days  
  
last_rating  
  
last_reviewed_at
```

```
next_review_at
```

```
favorite
```

```
created_at
```

```
updated_at
```

Unique:

```
user_id + card_id
```

review_events

```
id
```

```
user_id
```

```
card_id
```

```
rating
```

```
previous_interval_days
```

```
next_interval_days
```

```
reviewed_at
```

user_settings

```
user_id
```

```
daily_new_cards
```

```
daily_review_limit
```

```
created_at
```

```
updated_at
```

На первоначальном этапе этого достаточно.

31. Будущая миграция карточек в PostgreSQL

Когда JSON перестанет быть удобным:

создать таблицы:

```
topics
sections
cards
```

Написать импорт:

```
JSON → PostgreSQL
```

Так как `card.id` стабильный, существующий пользовательский прогресс не потеряется.

После миграции просто заменить:

```
StaticCardRepository
```

на:

```
SupabaseCardRepository
```

Это должно быть предусмотрено архитектурой заранее.

32. Telegram

Приложение в итоге должно работать как Telegram Mini App.

Telegram используется для:

- точки входа;
 - определения пользователя;
 - запуска приложения;
 - mobile/desktop контейнера;
 - получения Telegram theme.
-

33. Telegram Bot

Создать Telegram Bot.

Основной сценарий:

```
/start
```

Бот предлагает:

 **Открыть DevRecall**

При нажатии открывается Mini App.

Никакого сложного чат-бота на первом этапе не нужно.

Бот — точка входа в приложение.

34. Авторизация

Никаких:

- email;
- паролей;
- регистрации;
- формы входа.

Пользователь уже авторизован Telegram.

Frontend передаёт серверу Telegram Mini App init data.

Backend обязан проверить подлинность Telegram init data перед использованием Telegram user ID.

Нельзя считать `initDataUnsafe` достаточным доказательством личности пользователя.

35. Backend

Не создавать отдельный большой Node.js проект.

Использовать небольшие TypeScript serverless API functions внутри проекта.

Они нужны для:

- проверки Telegram init data;
 - определения пользователя;
 - записи прогресса;
 - получения прогресса;
 - записи review events;
 - работы с Supabase.
-

36. Секреты

Никогда не помещать во frontend:

- Telegram Bot Token;
- Supabase Secret Key;
- service-role ключ;
- другие административные secrets.

Они должны находиться только в environment variables backend.

37. Работа вне Telegram

Приложение должно запускаться обычной ссылкой.

Это нужно:

- для разработки;
- для debugging;
- для desktop browser.

В development режиме разрешено использовать локального тестового пользователя.

Production-логика должна различать:

Telegram environment

и

normal browser development

38. Frontend stack

Использовать:

- Vue 3
- TypeScript
- Composition API
- Vite
- Vue Router
- Pinia
- SCSS или обычный CSS
- Telegram Mini Apps WebApp API

- небольшая библиотека иконок

TypeScript:

```
strict: true
```

39. Почему здесь нужен Pinia

Pinia использовать только для нормального глобального состояния:

- текущий пользователь;
- текущая study session;
- topics;
- card progress;
- настройки.

Не переносить каждую локальную переменную компонента в store.

40. Предлагаемая структура проекта

```
dev-recall/  
  
src/  
  
  components/  
    common/  
    cards/  
    statistics/  
    topics/  
  
  views/  
    HomeView.vue  
    TopicsView.vue  
    TopicView.vue  
    StudyView.vue  
    FavoritesView.vue  
    StatisticsView.vue  
    SettingsView.vue  
  
  stores/  
    app.ts  
    study.ts  
    progress.ts  
  
  composables/
```

```
useStudySession.ts
useTelegram.ts

content/
  topics.json
  cards/

repositories/
  cards/
    CardRepository.ts
    StaticCardRepository.ts
    SupabaseCardRepository.ts

  progress/
    ProgressRepository.ts
    LocalProgressRepository.ts
    ApiProgressRepository.ts

services/
  study/
    reviewAlgorithm.ts
    sessionBuilder.ts

  telegram/
    telegram.ts

types/
  card.ts
  topic.ts
  progress.ts
  review.ts

utils/

router/

styles/

api/
  auth/
  progress/
  review/

scripts/
  validate-cards.ts
  content-stats.ts
```

Это ориентир, а не требование плодить пустые файлы.

Не создавать файлы и слои только потому, что они перечислены выше.

41. Настройки пользователя

Минимальная версия:

Новых карточек в день

Например:

10
20
30
50
без ограничения

Максимум повторений

Например:

50
100
без ограничения

Тема

- системная;
- светлая;
- тёмная.

В дальнейшем можно расширять.

42. Дополнительный функционал, который предусмотреть, но НЕ делать первым

Поиск

По вопросам и ответам.

Фильтры

- тема;
- раздел;
- уровень;
- новые;

- сложные;
 - изученные;
 - избранные.
-

Mock Interview

Отдельный режим собеседования.

Пользователь получает вопросы без сразу доступного ответа.

Например:

В вашем резюме указано, что вы использовали hybrid search. Почему одного vector search было недостаточно?

После ответа можно открыть эталон.

AI Interviewer

Когда базовый продукт уже стабилен:

пользователь отвечает текстом или голосом;

LLM оценивает ответ;

показывает:

- что правильно;
- что пропущено;
- насколько ответ соответствует уровню middle;
- уточняющий вопрос.

На первом этапе AI НЕ реализовывать.

43. Что сейчас категорически НЕ нужно

Не реализовывать на первой итерации:

- полноценную админку;
- LLM;
- voice recognition;
- сложный backend framework;
- Docker;
- Kubernetes;
- WebSocket;

- RabbitMQ;
- microservices;
- SSR;
- Nuxt;
- собственную дизайн-систему;
- сложные красивые графики;
- социальные функции;
- рейтинги пользователей;
- achievements на 100 вариантов;
- платёжную систему.

Это персональный тренажёр.

Не превращать его в стартап.

44. Этапы реализации

Этап 1 — фундамент

Создать:

- Vue 3 + TypeScript + Vite;
- Router;
- Pinia;
- базовые стили;
- responsive layout;
- нижнюю навигацию;
- основные routes.

Создать небольшой визуальный prototype.

Главная должна уже выглядеть как мобильное приложение.

Не заниматься Telegram.

45. Этап 2 — модель контента

Создать:

- Topic;
- Section;
- Card;
- CardProgress;
- ReviewEvent.

Создать:

`StaticCardRepository`.

Добавить примерно 20–30 тестовых карточек.

Не нужно пока генерировать сотни.

46. Этап 3 — основной UX

Реализовать:

- Home;
- Topics;
- Topic;
- Study;
- Favorites;
- Statistics.

Пока использовать mock/local data.

Главная задача:

полностью пройти пользовательский сценарий от открытия приложения до окончания study session.

47. Этап 4 — study engine

Реализовать:

- новые карточки;
- due cards;
- Again;
- Hard;
- Good;
- Easy;
- повтор Again внутри сессии;
- интервалы;
- difficult cards;
- progress;
- completion screen.

Алгоритм отдельно от Vue.

48. Этап 5 — local persistence

Добавить:

`LocalProgressRepository` .

После перезагрузки браузера должны сохраняться:

- прогресс;
- избранное;
- статистика;
- review history;
- настройки.

49. Этап 6 — статистика

Реализовать полноценный экран:

- today;
- week;
- streak;
- overall;
- topics;
- difficult questions.

Проверить корректность расчётов.

50. Этап 7 — Supabase

Только после того, как локальная версия работает хорошо.

Создать PostgreSQL.

Создать таблицы:

- users;
- card_progress;
- review_events;
- user_settings.

Создать API.

Локальный repository оставить для development.

51. Этап 8 — Telegram Mini App

Добавить:

- Telegram WebApp initialization;
- Telegram theme;
- safe areas;
- viewport;
- Telegram user;
- init data;
- backend validation;
- ApiProgressRepository.

Проверить:

- iPhone;
- Telegram Desktop.

Прогресс между ними должен синхронизироваться.

52. Этап 9 — Telegram Bot

Создать минимального бота.

`/start`

→ кнопка:

Открыть DevRecall

Mini App открывается внутри Telegram.

53. Этап 10 — deployment

Frontend + serverless functions:

Vercel

Database:

Supabase

Git repository должен быть подключён к Vercel.

Push в main должен автоматически обновлять приложение.

54. Этап 11 — наполнение реальными карточками

Только теперь начинать массово добавлять учебный контент.

Первая большая партия должна быть непосредственно связана с подготовкой пользователя к собеседованиям.

Приоритет:

1. JavaScript
2. Vue
3. SQL / PostgreSQL
4. PHP / Laravel
5. вопросы по резюме
6. REST / backend
7. Security
8. Git
9. Docker / CI/CD
10. RAG / LLM
11. RabbitMQ / MinIO / Kubernetes и остальные технологии

Не пытаться сразу создать 2000 карточек.

Лучше качественные наборы по 30–100 карточек на тему.

55. Требования к качеству карточек

Карточки не должны превращаться в энциклопедию.

Одна карточка преимущественно проверяет одну конкретную мысль.

Плохо:

Расскажи всё про Event Loop.

Хорошо:

Что произойдёт раньше: `Promise.then()` или `setTimeout(..., 0)` и почему?

Также нужны разные типы вопросов:

Определение

Что такое closure?

Почему

Почему closure продолжает иметь доступ к переменной внешней функции?

Код

...

Что выведет этот код?

Практика

Где closure используется в реальном приложении?

Собеседование

В каких случаях использование closure может привести к проблемам с памятью?

По резюме

В резюме написано, что вы устраняли N+1. Как вы обнаружили проблему и как исправили?

56. Важное требование к AI-агенту

Не менять архитектуру и стек без причины.

Если возникает новая идея:

сначала определить, действительно ли она нужна для текущей версии.

Приоритет проекта:

Работающее обучение

>

удобство

>

статистика

>

синхронизация

>

красота

>

дополнительные функции

57. Git

Работать небольшими логическими изменениями.

Желательно отдельный commit после каждого законченного этапа.

Примеры:

```
feat: initialize Vue application
feat: add mobile navigation
feat: add card repository
feat: implement study session
feat: add review algorithm
feat: add favorites
feat: add statistics
feat: integrate Telegram Mini App
```

Не делать один огромный commit на весь проект.

58. README

Параллельно поддерживать README.

В нём должны быть:

- что такое DevRecall;
- стек;
- архитектура;
- запуск локально;
- environment variables;
- структура карточек;
- как добавить новую тему;
- как добавить карточки;
- как проверить карточки;
- deployment;
- Telegram integration.

Особенно подробно:

Как добавить новые карточки

Чтобы позже пользователь мог просто передать эту инструкцию AI-агенту.

59. Definition of Done первой полноценной версии

Версия считается готовой, когда пользователь может:

- открыть приложение в Telegram на iPhone;
- открыть его в Telegram Desktop;
- иметь единый прогресс;
- выбрать тему;
- начать обучение;
- увидеть вопрос;
- открыть ответ;
- нажать Не знаю / Сложно / Нормально / Легко;
- получить следующую карточку;
- добавить вопрос в избранное;
- учить избранные;
- видеть сложные карточки;
- завершить сессию;
- посмотреть результат;
- открыть статистику;
- увидеть серию дней;
- увидеть прогресс по темам;
- закрыть Telegram;
- вернуться позже;
- продолжить с сохранённого состояния.

Также AI-агент должен иметь возможность добавить новую пачку карточек без изменения Vue-компонентов.

60. Главный принцип проекта

DevRecall создаётся не ради разработки DevRecall.

Он создаётся ради подготовки к собеседованиям.

Поэтому после достижения полноценного MVP основная работа должна идти не в увеличение количества функций приложения, а в:

- изучение тем;
- добавление качественных вопросов;
- прохождение карточек;
- обнаружение слабых мест;
- добавление новых вопросов по этим слабым местам.

Если новая функция не помогает пользователю лучше учиться или запоминать материал — её приоритет низкий.

61. С чего начать агенту прямо сейчас

Не пытайся выполнить всё ТЗ одним большим изменением.

Начни только с **Этапа 1**.

Нужно:

1. Инициализировать Vue 3 + TypeScript + Vite.
2. Подключить Vue Router и Pinia.
3. Создать базовую структуру проекта.
4. Создать mobile-first layout.
5. Сделать bottom navigation.
6. Сделать визуальные прототипы:
 7. Home;
 8. Topics;
 9. Favorites;
 10. Statistics.
 11. Добавить временные mock-данные.
 12. Добиться ощущения нормального мобильного приложения.
 13. Проверить desktop layout.
 14. Запустить typescript и build.

После этого остановиться.

Не начинать БД, Telegram, алгоритм повторения и server API, пока первый визуальный фундамент не будет просмотрен и принят пользователем.